

Chaine CI/CD et deploiement (MSPR3 / TPRE601)

Industrialisation de la plateforme HealthAI Coach. Chaque service possede sa propre chaine d'integration continue (GitHub Actions) qui teste le code puis publie une image Docker sur GitHub Container Registry (GHCR). Le deploiement se fait en local via `bootstrap.sh`, qui orchestre l'ensemble avec Docker Compose.

Vue d'ensemble du flux

```
commit/push (branche par default ou tag vX.Y.Z)
|
v
GitHub Actions (un workflow par depot)
|-- lint (selon la techno)
|-- tests + couverture
|-- build de l'image Docker
v
publication sur GHCR (ghcr.io/whitefoxyt/mspr-<service>)
|
v
deploiement local : ./bootstrap.sh -> docker compose up -d
```

La publication de l'image n'a lieu que sur `push` (pas sur les pull requests) et seulement si le lint et les tests passent. Les workflows `docker-publish` sont appeles via `workflow_call` depuis le workflow de CI principal.

Chaine par service

Service	Depot (whitefoxyt)	Declencheurs	Lint	Tests	Image GHCR
API	MSPR-HealthAI-Coach-API	push/PR <code>main</code> , tags <code>v*.*.*</code>	non	JUnit + JaCoCo (seuil 80 %)	<code>mspr-api</code>
Auth	MSPR-HealthAI-Coach-AUTH	push/PR <code>main</code> , tags <code>v*.*.*</code>	non	<code>bun test</code>	<code>mspr-auth</code>
AI-Nutrition	MSPR-HealthAI-Coach-AI-Nutrition	push/PR <code>master</code> , tags <code>v*.*.*</code>	ruff	pytest + couverture	<code>mspr-ai-nutrition</code>
Reco-Fitness	MSPR-HealthAI-Coach-Reco-Fitness-	push/PR <code>master / main</code> , tags <code>v*.*.*</code>	ruff	pytest-cov (seuil 80 %) avec services PostgreSQL + MongoDB	<code>mspr-reco-fitness</code>
ETL	MSPR-HealthAI-Coach-ETL	push/PR <code>master / main</code> , tags <code>v*.*.*</code>	ruff + format	pytest unitaires + integration (PostgreSQL)	<code>mspr-etl</code>
Front	MSPR-HealthAI-Coach-Dahsboard	push/PR <code>main</code> , tags <code>v*.*.*</code>	oxlint + eslint + vue-tsc	Vitest + Cypress (e2e)	<code>mspr-front</code>
BDD	MSPR-HealthAI-Coach-BDD	push/PR <code>main</code> , tags <code>v*.*.*</code>	non	validation des migrations SQL (PostgreSQL)	<code>mspr-db</code>
MongoDB	MSPR-HealthAI-Coach-MongoDB	push/PR <code>master / main</code> , tags <code>v*.*.*</code>	<code>node -check</code>	validation des collections et index (MongoDB)	<code>mspr-mongodb</code>

Strategie d'images et de tags

Les workflows `docker-publish` utilisent `docker/metadata-action` et produisent plusieurs tags par image :

- `type=ref,event=branch` : tag au nom de la branche.
- `type=semver,pattern={{version}}` et `{{major}}.{{minor}}` : sur les tags Git `vX.Y.Z`.
- `type=sha` : SHA court du commit, pour la tracabilite.
- `type=raw,value=latest,enable={{is_default_branch}}` : `latest` uniquement depuis la branche par default du depot.

Le deploiement en mode prod (`bootstrap.sh` sans option) tire les images `:latest` , donc celles publiees depuis la branche par default de chaque depot.

Qualite de code

- **Tests automatisés** sur chaque service (JUnit, bun test, pytest, Vitest/Cypress, validations SQL et MongoDB). Seuil de couverture à 80 % impose cote API (JaCoCo) et Reco-Fitness (pytest-cov).
- **Linting** selon la techno : ruff (Python), oxlint + eslint + vue-tsc (Front), `node --check` (MongoDB).
- **SonarQube** : le service API embarque le plugin Sonar (`pom.xml` , organisation `healthai-coach` , projet `healthai-coach-api`) et un environnement d'analyse local (`docker-compose.sonar.yml` dans le depot API) pour une analyse manuelle (`mvn sonar:sonar`). La CI de l'API contient en plus une étape SonarCloud qui se déclenche automatiquement dès qu'un secret GitHub `SONAR_TOKEN` est configuré (sinon elle est ignorée, sans casser le pipeline). Pour l'activer : créer le projet sur SonarCloud sous l'organisation `healthai-coach` , puis ajouter le token en secret `SONAR_TOKEN` du depot.

Deploiement

Le déploiement de la plateforme est décrit en détail dans `README.md` . En résumé :

- **Mode prod** : `./bootstrap.sh` tire les images GHCR `:latest` et lance `docker compose up -d` . C'est l'étape de mise en production locale (cible d'évaluation et de demo).
- **Mode dev** : `./bootstrap.sh --dev` clone les 8 depots sources et build les images localement (`docker-compose.dev.yml`).

`bootstrap.sh` prépare aussi l'environnement : création du `.env` , génération des secrets (`BETTER_AUTH_SECRET` , mots de passe des bases), déchiffrement des clés API.

Trois configurations sont disponibles via des overlays Compose (voir `CONFIGS.md`) : complete (avec monitoring), offline (sans internet) et performance (limites de ressources).

Sauvegarde, restauration, supervision

- Sauvegarde et restauration des bases : `scripts/backup.sh` , `scripts/restore.sh` (voir `README.md`).
- Observabilité : overlay `docker-compose.monitoring.yml` (Prometheus, Grafana, Loki, Alertmanager + exporters). Voir `monitoring/README.md` .